

WEEK 1

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- Write a C code to create a process using fork() system call.
- Read the user input as shell command.
- Use a system call exec() to execute the shell command.
- Use a wait() system call for waiting a parent process.
- Display the PIDs of parent and child processes.

1) Create a file

```
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu\home\saichaitanya> wsl
saichaitanya@GeethzzSai:~$ nano main.c
saichaitanya@GeethzzSai:~$ |
```

Then press enter to open command

2)write a code

```
shivam@MSH23: ~
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());

        execl("/bin/sh", "sh", "-c", command, NULL);

        // Executes only if execl fails
        perror("Execution failed");
        exit(1);
    }

    else {
        // Parent Process
        printf("\nParent Process\n");

        [ Read 46 Lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^G Location   ^U Undo       ^M Set Mark   ^- To Bracket
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^E Redo       ^C Copy       ^Q Where Was
```

```

else {
    // Parent Process
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}

```

- 2) To save press control O
- 3) To come back to shell press control X
- 4) Then for compilation write a command
gcc filename.c -o filename

```

saichaitanya@GeethzzSai: ~/Practical_week2
saichaitanya@GeethzzSai:~/Practical_week2$ cd src
saichaitanya@GeethzzSai:~/Practical_week2/src$ nano main.c
saichaitanya@GeethzzSai:~/Practical_week2/src$ gcc main.c -o main

```

Then press enter then to view a output write a command
./filename

```

saichaitanya@GeethzzSai:~/Practical_week2/src$ nano main.c
saichaitanya@GeethzzSai:~/Practical_week2/src$ gcc main.c -o main
saichaitanya@GeethzzSai:~/Practical_week2/src$ ./main
Enter Linux command: |

```

Then you will be asking to enter linux command to view the output
In ENTER LINUX COMMAND: enter either "pwd" command (or) "ls" command
to see the output of your code

```
Child PID: 977
saichaitanya@GeethzzSai:~/Practical_week2/src$ nano main.c
saichaitanya@GeethzzSai:~/Practical_week2/src$ gcc main.c -o main
saichaitanya@GeethzzSai:~/Practical_week2/src$ ./main
Enter Linux command: pwd

Parent Process
Parent PID: 974

Child Process
Child PID: 977
/home/saichaitanya/Practical_week2/src
Child process completed.
saichaitanya@GeethzzSai:~/Practical_week2/src$ |
```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
Child process completed.
saichaitanya@GeethzzSai:~/Practical_week2/src$ uname
Linux
saichaitanya@GeethzzSai:~/Practical_week2/src$ |
```

child process completed:

```
saichaitanya@GeethzzSai:~/Practical_week2/src$ uname  
Linux
```

```
saichaitanya@GeethzzSai:~/Practical_week2/src$ lscpu
```

```
Architecture:          x86_64  
CPU op-mode(s):        32-bit, 64-bit  
Address sizes:          39 bits physical, 48 bits virtual  
Byte Order:             Little Endian  
CPU(s):                 12  
On-line CPU(s) list:    0-11  
Vendor ID:              GenuineIntel  
Model name:             13th Gen Intel(R) Core(TM) i5-1334U  
CPU family:             6  
Model:                  186  
Thread(s) per core:     2  
Core(s) per socket:     6  
Socket(s):              1  
Stepping:               3  
BogoMIPS:               4992.00  
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtr  
r pge mca cmov pat pse36 clflush mmx fxsr sse s  
se2 ss ht syscall nx pdpe1gb rdtscp lm constant  
_tsc rep_good nopl xtopology tsc_reliable nonst  
op_tsc cpuid tsc_known_freq pni pclmulqdq vmx s  
sse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe p  
opcnt tsc_deadline_timer aes xsave avx f16c rdr  
and hypervisor lahf_lm abm 3dnowprefetch ssbd i  
brs ibpb stibp ibrs_enhanced tpr_shadow ept vpi  
d ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi  
2 erms invpcid rdseed adx smap clflushopt clwb  
sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni  
vnmi umip waitpkg gfni vaes vpclmulqdq rdpid mo  
vdiri movdir64b fsrm md_clear serialize ibt flu  
sh_lld arch_capabilities  
  
Virtualization features:  
Virtualization:         VT-x  
Hypervisor vendor:      Microsoft  
Virtualization type:    full
```

```

Virtualization features:
  Virtualization:      VT-x
  Hypervisor vendor:   Microsoft
  Virtualization type: full
Caches (sum of all):
  L1d:                 288 KiB (6 instances)
  L1i:                 192 KiB (6 instances)
  L2:                  7.5 MiB (6 instances)
  L3:                  12 MiB (1 instance)
NUMA:
  NUMA node(s):        1
  NUMA node0 CPU(s):   0-11
Vulnerabilities:
  Gather data sampling: Not affected
  Ghostwrite:           Not affected
  Indirect target selection: Not affected
  Itlb multihit:        Not affected
  L1tf:                 Not affected
  Mds:                  Not affected
  Meltdown:             Not affected
  Mmio stale data:      Not affected
  Old microcode:        Not affected
  Reg file data sampling: Mitigation; Clear Register File
  Retbleed:             Mitigation; Enhanced IBRS
  Spec rstack overflow: Not affected
  Spec store bypass:    Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:           Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:           Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI BHI_DIS_S
  Srbds:                Not affected
  Tsa:                  Not affected
  Tsx async abort:      Not affected
  Vmscape:              Not affected
saichaitanya@GeethzzSai:~/Practical_week2/src$

```

```

Srbds:                Not affected
Tsa:                  Not affected
Tsx async abort:      Not affected
Vmscape:              Not affected
saichaitanya@GeethzzSai:~/Practical_week2/src$ ps
  PID TTY          TIME CMD
   860 pts/0        00:00:00 bash
  1051 pts/0        00:00:00 ps
saichaitanya@GeethzzSai:~/Practical_week2/src$ |

```

```

1031 pts/0    00:00:00 ps
saichaitanya@GeethzzSai:~/Practical_week2/src$ top
top - 03:47:49 up 41 min, 1 user, load average: 0.02, 0.02, 0.00
Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.
MiB Mem :   7784.6 total,   6992.9 free,    582.8 used,    360.0 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,     0.0 used.   7201.8 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1	root	20	0	24268	15340	11540	S	0.0	0.2	0:00.88
2	root	20	0	3180	2212	2072	S	0.0	0.0	0:00.01
6	root	20	0	3212	2148	2008	S	0.0	0.0	0:00.09
48	root	19	-1	50396	16832	15552	S	0.0	0.2	0:00.32
83	systemd+	20	0	22540	14628	12004	S	0.0	0.2	0:00.11
89	root	20	0	35424	12356	9204	S	0.0	0.2	0:01.00
109	root	20	0	2888	1948	1820	S	0.0	0.0	0:00.07
110	root	20	0	4472	3168	2924	S	0.0	0.0	0:00.02
111	message+	20	0	8820	5452	4780	S	0.0	0.1	0:00.10
115	root	20	0	44092	29872	14728	S	0.0	0.4	0:00.37
119	root	20	0	18436	9420	8172	S	0.0	0.1	0:00.12
186	syslog	20	0	220548	5560	4624	S	0.0	0.1	0:00.13
196	root	20	0	5492	2736	2556	S	0.0	0.0	0:00.02
198	root	20	0	123456	32644	18036	S	0.0	0.4	0:00.31
226	_chrony	20	0	24444	11928	7152	S	0.0	0.1	0:00.16
232	_chrony	20	0	12092	2448	1804	S	0.0	0.0	0:00.02
333	root	20	0	8648	5616	4772	S	0.0	0.1	0:00.02
377	saichai+	20	0	22332	13604	10984	S	0.0	0.2	0:00.17
379	saichai+	20	0	22908	4116	2116	S	0.0	0.1	0:00.00
407	saichai+	20	0	6136	5404	3860	S	0.0	0.1	0:00.03
855	root	20	0	3184	1124	980	S	0.0	0.0	0:00.00
857	root	20	0	3200	1260	1108	S	0.0	0.0	0:00.08
860	saichai+	20	0	6268	5544	3872	S	0.0	0.1	0:00.11
1017	root	20	0	1792044	15116	12560	S	0.0	0.2	0:00.03
1063	saichai+	20	0	8172	6004	3900	R	0.0	0.1	0:00.00